

Backend Migration Report

adVantage – from a local-first SQLite WASM app to a multi-user Express and PostgreSQL deployment

Completed 19 September 2026 · follows the Backend Migration Roadmap of the same date

The app is rebuilt on the new stack and working. Anyone can sign up, and each account sees its own ledger and nobody else's. Every screen, chart and figure from the local-first build is present and reaches the same numbers.

The roadmap predicted that the database swap would be the cheap part, because the query layer was already written as functions that each take a database handle as their first argument. That held. Those functions now take a **tenant handle** – `{ db, userId }` – in exactly that position, and the screens on top of them went from `listAccounts(db, ...)` to `listAccounts(api, ...)` and otherwise did not change. That symmetry is what kept seven thousand lines of interface out of the blast radius.

What moved, what did not

AREA	SIZE	DISPOSITION
packages/core	940 lines	Byte-identical. Money, dates, taxonomy, analytics, schedule expansion. Verified with a recursive diff against the old repo. It is the reason this was a port and not a rewrite.
packages/theme	370 lines	Byte-identical. Design tokens and the validated chart palette.
Query layer	46 functions	43 ported behind the tenant handle, plus three the server needed: <code>loadDashboard</code> , <code>loadOutlook</code> and <code>resetLedger</code> .
Schema	11 models	Rewritten for Postgres. Seven ledger tables, each carrying <code>user_id</code> from the first migration, and four Better Auth tables in the same database.
Raw SQL	2 fragments	Down from 27. Prisma's typed API expresses the rest, which removed the whole class of unqualified-column faults the old code had to guard against by hand.
SQLite worker, bridge, migrator	~600 lines	Deleted. Replaced by a connection pool and Prisma Migrate.
apps/web	7,215 lines	Kept. 36 of 59 source files are byte-identical; 20 changed by 776 lines in total, six of them by exactly the two lines of an import swap. Three files are new: the API client, the session provider and the sign-in screen.
apps/api	3,024 lines	New. Express service: routes, the ported query layer, validation, auth wiring and the Prisma schema.

Your own records

Because money stayed an integer count of centavos and dates stayed `YYYY-MM-DD` text, the backup format is unchanged – a file downloaded from the offline build imports here without translation. This was verified with the real one: **7 accounts, 42 categories, 37 planned payments and 2 income sources** loaded intact, with the outlook projecting 84 entries off the imported schedule.

The stack, as built

LAYER	CHOICE
Monorepo	Turborepo + npm workspaces, unchanged
Frontend	Vite, React 19, TypeScript strict, Tailwind v4, shadcn-style components on Radix, lucide, Recharts – unchanged
Backend	Express 5.2 on Node, TypeScript strict, Zod at every write
Database	PostgreSQL 17 (Docker Compose for development, on host port 5433), Prisma 6.19
Identity	Better Auth 1.7, its tables inside the same migrations
Transport	REST over <code>fetch</code> , typed end to end through a shared contract package
State	Native React hooks only – still no data-fetching or state library

The roadmap suggested tRPC. Express with a typed REST surface was chosen instead because that was the stack asked for, and the contract is shared rather than inferred: `packages/api-client` holds the wire types and both sides import them, so a field renamed on the server fails to compile in the browser rather than arriving as `undefined` at runtime.

How tenancy is enforced

Every application row carries `user_id`, written in the first migration rather than retrofitted. Three things hold the line, in descending order of how much weight they carry:

- **The tenant handle.** Query functions cannot be called without naming a user, and route code cannot reach the unscoped Prisma client at all – it is exported from one module that `routes/` does not import.
- **Scoped writes.** Updates and deletes go through `updateMany` and `deleteMany` with `userId` in the `where` clause, so one person's id can never address another person's row even when it is guessed correctly.
- **Cascade from the user row.** Deleting an account deletes the records rather than orphaning them. Verified: removing ten test users left zero rows across all eight tables.

Not yet in place: row-level security

The roadmap wanted Postgres RLS so that application filtering would be the fast path rather than the only defence. It is **not** implemented. Doing it properly under Prisma means `SET LOCAL app.user_id` inside an interactive transaction per request, which changes the connection model, so it was deferred rather than half done. Until it lands, the scoping above is the only thing between two users' data – which is why it is tested at the query layer and again over HTTP.

The one the roadmap singled out

`importBackup` deleted every row in every table before inserting. Unscoped in a multi-user world that is a cross-tenant wipe triggered by a file upload. It now deletes only within the caller's account, inside a transaction, and there is a test that has one account restore an empty backup and then checks that another account's row counts are untouched.

Against the roadmap

#	PHASE	STATUS	NOTES
0	Prepare and decide	Done	Postgres in Docker rather than Neon, for development. Better Auth chosen over Clerk, as recommended.
1	Tenancy and auth spike	Done	Schema with <code>user_id</code> , tenant handle, Better Auth, isolation tests. RLS deferred.
2	Port the query layer	Done	All 43 functions. Four integer-boolean columns became real booleans. Money stayed integer centavos; client-generated UUIDs kept.
3	API layer	Done	41 routes. One endpoint per heavy screen rather than request batching, which is the same fix by a different route.
4	Rewire the client	Done	Every screen works against the API. Optimistic writes not added – see the deferred list.
5	Accounts and deletion	Partial	Sign-up, sign-out, per-user export and real deletion all work. No invite gate (sign-up is open by request) and no password reset.
6	Deploy and first invite	Not started	Deployment notes are in the README; nothing is hosted yet.
7	After the first users	Not started	Correctly so – there are none.

Gate after phase 1: the isolation test passes, at both the query layer and over HTTP. **Gate after phase 4:** adding a record still feels immediate. **Gate after phase 6:** not reached – do not send an invite before restoring a backup into a scratch database at least once.

Reads used to cost about 0.2 ms, so the app refetched everything after every write and never had to think about it. Three consequences surfaced, each caught by a test rather than by reasoning:

1. The dashboard was twelve queries on load

Free against a local file, a thundering herd over a network. The fan-out still happens, but next to the database:

`loadDashboard` and `loadOutlook` assemble whole screens server-side and the browser makes one request each. The derivation is still `packages/core`, unchanged – it simply runs somewhere else now.

2. Stale figures under a fresh heading

Switching the Outlook timeframe re-rendered the *previous* window's rows under the *new* caption, because the caption is local state and the rows are a round trip. The browser test caught it counting 46 items that belonged to a timeframe no longer selected.

`useLiveQuery` now reports `stale` – a request in flight for a different question than the one the data on screen answers. Re-running the same query after a write and briefly showing the previous figures is a courtesy; changing the month and showing last month's figures under this month's heading is a lie. The Outlook card and the dashboard check for it.

3. A 409 in the console, seconds after sign-up

The first page load fires several requests at once. On a brand new account they all found an empty ledger and all tried to seed it; the losers failed on the slug unique index. Fixed with a Postgres advisory lock keyed on the user, taken inside the seeding transaction – which holds across processes in a way an in-memory flag cannot – plus in-process de-duplication so the other requests do not each open a transaction to wait on it.

SUITE	CHECKS	WHAT IT PROVES
packages/core	32	Money parsing and formatting, month arithmetic, recurrence, breakdown folding, budget pace. Unchanged from the offline build and still passing, which is the evidence that the arithmetic did not move.
apps/api data layer	44	Real Postgres, real migrations, real Prisma. The same assertions the SQLite build made – seeding, joins, derived balances, aggregates, group budgets counting their subcategories, posting and un-posting, the outlook double-count guard, backup and import – plus a new section on tenant isolation.
API over HTTP	45	Sign-up, the first-run seed, the whole ledger, and the isolation model end to end: one account pointed at every one of another's row ids, reading, updating, deleting and restoring an empty backup over them. Nothing gets through.
Built app in Chrome	62	Sign-up through the real form, the session cookie surviving a reload, an expense and an income saved, cash flow bars pointing the right way, a plan file importing without moving net worth, Outlook in all three timeframes, the wallet with uniformly sized marks, every route, sign-out, and a clean console.

Typecheck and build are clean across all six workspaces. The data-layer tests create their own throwaway accounts and delete them afterwards, which is safe precisely because of the tenancy they are testing.

Deliberately not done

ITEM	WHY, AND WHAT IT WOULD TAKE
Row-level security	Changes the connection model under Prisma. The largest outstanding item and the one to do first.
Email verification and password reset	There is no transactional sender wired. A verification mail that never arrives locks people out of their own records, so the setting is off rather than broken.
Invite gate	Sign-up is open, by request. The roadmap assumed invite-only; that assumption was changed deliberately, not overlooked.
Optimistic writes	Adding a record already feels immediate on a local network. Client-generated UUIDs are kept so this stays available when latency makes it worth doing.
Deployment	Notes are in the README. Nothing is hosted, no migrations run in CI, and no backup has been restored into a scratch database – which the roadmap makes a hard gate before inviting anyone.

Running it

```
npm run setup      # install, write .env with a generated secret, start Postgres, migrate
npm run dev        # API on :4000, app on http://localhost:5173
```

Every command here is one line on purpose. Windows PowerShell 5.1 has no `&&` and does not read a trailing backslash as a line continuation, so commands written the usual way fail there with *The token '&&' is not a valid statement separator*. Inside an npm script `&&` is fine, because npm runs those through `cmd.exe`.

To load the existing records into an account – sign up in the app first, so the account exists:

```
npm run import:backup -w @advantage/api -- --email you@example.com --file
./initial_data/advantage-backup-2026-09-19.json
```

It refuses to overwrite a ledger that already has records unless given `--force`, and writes a safety copy first when it does.

What was traded away

The old app could say honestly that records never left the browser. That is over, and it is worth naming rather than discovering later. What softens it is the export path, which is why it stayed a first-class button in Settings rather than becoming a menu item: the trust story is now "you can take everything out whenever you like", and account deletion genuinely deletes.